

Fog of War without Zero-Knowledge Proofs: A Blinded Sighting Test for Trustless Game Channels

Andrew Colosimo Roy Crombleholme Andrew Gore
Konstantin Gorskov Daniel Kraft

Xaya Developers

September 14, 2026

Abstract

A game channel must let players discover encounters without distributing all hidden positions. We describe a deterministic instantiation of classical Diffie–Hellman private membership testing and its use with signed transcripts, epoch disclosures and bounded dispute checks. The standalone implementation computes one directed membership result per party when both follow the protocol. We prove conditional correctness, specify its byte format, and test the C++ implementation against an independent BigInt implementation. Privacy against active deviations is a separate question: fabricated queries can reveal information before an audit, and later punishment cannot undo that disclosure. The contribution is an implementation and integration case study, not a new PSI primitive or a proof of malicious security. We distinguish this reference exchange from Dungeon Channel’s newer sealed-envelope sight delivery and identify the remaining proof and measurement requirements.

1 Scope and contribution

A commitment to a hidden position does not itself tell another player whether that position is visible. Sending every position defeats fog of war; withholding all of them prevents encounters. Private membership testing supplies an intermediate operation: a client learns whether its position belongs to another player’s visible set. Commitments, signatures and an eventual reveal can then support checks against game rules. Salted hash commitments provide *computational* hiding under appropriate assumptions and secret entropy, not perfect hiding.

This paper specifies the sight-only exchange of the reference library [7], implemented in `include/fow.hpp` and `src/fow.cpp`. The library accepts element codes and sets; it does not implement a map, signed channel, wallet, seed lifecycle, hearing block or game referee. Its frozen fixture is the output of Dungeon Channel revision `1ad3b7f` of 2026-08-01 14:54 UTC, a revision the retained history of [8] no longer contains: the omnidirectional radius-6 line-of-sight set of spawn 0 on map seed `0x5EED1234`, computed before the game applied a facing beam (Appendix A). It is not a claim of byte compatibility with the current game’s complete wire format (§5).

The contribution is the concrete combination of deterministic per-round blinding, rotating padding, signed transcript commitments, and game-specific checks over disclosed epochs. Each ingredient, and the combination itself in outline, has precedent (§6); what is specific here is the per-round transcript format with seed-derived, rotating padding, its implementation and its tests. In particular, neither PSI-based fog of war nor detecting cheating after execution is new. We

make no claim to be the first such game or to introduce a new cryptographic security model. The reproducible artifact and its explicit integration constraints are the basis for assessing the narrower engineering contribution.

2 The reference exchange

Inputs and outputs. Party A has a secret element a and a set V_A ; B has b and V_B . The intended outputs are $[a \in V_B]$ to A and $[b \in V_A]$ to B . These results need not agree. For example, a directional sight cone can contain an opponent whose own cone points away. Agreement follows only when the relation is symmetric. The test does not reveal an element’s plaintext value as part of its output, but that is not by itself a privacy theorem.

For a grid game one can define

$$V(w, f) = \{t : d(w, t) \leq \rho, \text{ los}(w, t), \text{ beam}_f(w, t)\},$$

where d is walkable path distance and f is a committed facing. For a four-connected grid, the path-radius ball is contained in an ℓ_1 ball of size $1 + 2\rho(\rho + 1)$. At $\rho = 6$ this is 85, so a pad of 128 accommodates any such visible set, independently of the map. This is a bound, not a measurement of a particular generator.

Group and encoding. The implementation uses Curve25519 Montgomery u -coordinates over $p = 2^{255} - 19$, encoded in 32 little-endian bytes. The prime subgroup order is $\ell = 2^{252} + 2774231777372353535851937790883648493$. Its ladder takes the supplied scalar without X25519 clamping. Hash-to-point is SHA-256 followed by the x-only Elligator 2 map and multiplication by 8. Three conventions are load-bearing for byte reproduction. The digest is read as a little-endian integer with bit 255 cleared and is not reduced modulo p ; the nineteen values in $[p, 2^{255})$ act modulo p . The map is RFC 9380 Appendix F.3 with $Z = 2$: $x_1 = -A/(1 + 2u^2)$ is taken when $x_1^3 + Ax_1^2 + x_1$ is a square in $\mathbb{F}_p$, otherwise $x_2 = -x_1 - A$; for $u \neq 0$ exactly one candidate is a square. The result is multiplied by 8 with the unclamped ladder. Three inputs reach the identity that way: $u = 0$ maps to the order-2 point, and the two u with $1 + 2u^2 = -A$ map to the order-4 point $x = 1$; the exhaustive check of §5 confirms that no element code hits any of them. Point encodings are read the same way, with bit 255 cleared and no reduction, so 2^{255} and p are non-canonical encodings of zero (§4). The intent is to obtain points in the prime-order subgroup with unknown discrete logarithms [3, 4, 5]. This single-map construction is not an RFC 9380 uniform hash-to-curve suite [6]; that standard’s security results must not be imported without justification. Correctness below is conditional on nonidentity prime-order points and distinct point encodings for the relevant inputs. The privacy of this particular encoding requires separate analysis.

Deterministic derivations. Let s_i be a secret 32-byte seed for party i . Use an independent seed per party and match; an epoch or round must not reuse a previously exposed seed. For a round r and element code t the reference defines

$$\begin{aligned} P_t &= \text{H2C}(\text{DCHT} \parallel \text{LE16}(t)), \\ \alpha_{i,r} &= \text{mask}_{252}(\text{SHA-256}(\text{DCHP} \parallel s_i \parallel \text{LE32}(r))), \\ D_{i,r,j} &= \text{H2C}(\text{DCHD} \parallel s_i \parallel \text{LE32}(r) \parallel \text{LE16}(j)). \end{aligned}$$

The tags are raw four-byte ASCII strings without a terminator; LE16 and LE32 are little-endian 16- and 32-bit encodings. Here mask_{252} clears the high four bits of byte 31 of the little-endian

scalar, replacing zero by one. This is masking, not modular reduction or X25519 clamping. It produces a nonzero scalar below 2^{252} , hence below the subgroup order. The API accepts 16-bit element codes and a 16-bit round, although the round’s hash preimage occupies four bytes. A caller must not wrap the round under the same seed.

Set $N = 128$. Let $n_i \leq N$ be the number of codes party i supplies; duplicates are kept, so n_i is a count rather than a cardinality and V_i below is read as a list. The pad indices are $j = n_i, \dots, N - 1$. Real points and pad points are blinded together and sorted in ascending lexicographic order of their 32-byte encodings (`memcmp` order, not numeric order). The order in which the caller enumerates the real set does not affect the result. The private seeds still require secure random initialization; deterministic derivation does not eliminate that requirement.

Messages. Suppressing the round subscript, for $i \in \{A, B\}$ define

$$S_i = \text{sort}(\{\alpha_i P_t : t \in V_i\} \cup \{\alpha_i D_{i,r,j} : n_i \leq j < N\}), \\ Q_A = \alpha_A P_a, \quad Q_B = \alpha_B P_b.$$

The first flight sends (S_A, Q_A) from A to B and (S_B, Q_B) in the other direction. The second sends

$$A \longrightarrow B : R_A = \alpha_A Q_B, \quad B \longrightarrow A : R_B = \alpha_B Q_A.$$

Party A computes

$$b_A = [R_B \in \{\alpha_A s : s \in S_B\}],$$

and B computes the analogous result. This costs a scan of the padded set; there is no early exit in the C++ implementation. The comparison is byte equality of canonical encodings: the set entries pass through the ladder, which canonicalizes them, while R_B is compared as received, so a responder must send the canonical encoding.

Theorem 1 (Conditional honest correctness). *If both parties use the specified inputs and derivations, all point outputs are nonidentity points of the prime-order subgroup, the scalars are nonzero modulo its order, and the points derived from distinct real or dummy preimages used in the execution have distinct x-coordinates, then $b_A = [a \in V_B]$ and $b_B = [b \in V_A]$.*

Proof. For each real entry t in B ’s set, $\alpha_A(\alpha_B P_t) = \alpha_B(\alpha_A P_t)$. Multiplication by the nonzero product $\alpha_A \alpha_B$ is a bijection in the prime-order subgroup. With x-only encodings it is also a bijection on the classes $\{P, -P\}$. Thus its output equals R_B precisely when the input class equals that of P_a . The distinctness hypothesis excludes a false match from another real point or a domain-separated dummy. Consequently the scan succeeds precisely when $a \in V_B$. The other direction is identical. $\square$

The theorem establishes correctness, not privacy, input validity, or fairness. For the real universe the distinctness hypothesis is checked exhaustively: `make test` computes all 2^{16} element points and confirms that none is the identity and that no two share an x-coordinate. It remains a hypothesis for the seed-dependent dummies. Assigning a negligible failure probability there requires an explicit model of the hash-to-point distribution; a generic SHA-256 birthday bound alone is not a proof about this nonuniform map.

3 Transcript binding and game adjudication

What the library commits. `petFlightHash` constructs a binary Merkle tree with 256 leaves. Leaves 0–127 contain the sorted set, leaf 128 contains Q , leaf 129 contains R , and the remaining leaves contain 32 zero bytes. The hashes are

$$L_j = \text{SHA-256}(\text{DCHV} \parallel 00 \parallel \text{LE16}(j) \parallel \text{point}_j),$$

$$H(L, R) = \text{SHA-256}(\text{DCHV} \parallel 01 \parallel L \parallel R).$$

The prefixes are single bytes and separate leaves from internal nodes; the index binds a point to its slot. Nodes pair adjacent entries $(2m, 2m + 1)$ of each level with the lower index on the left, level by level up to the root. An opening carries eight sibling hashes, bottom level first. It proves membership in a committed transcript, not that the transcript was computed honestly.

The root must be authenticated by the surrounding channel. The bytes a client uses to make a decision must be the authenticated bytes that a later judge checks. Recomputing an honest transcript cannot prove what an opponent sent over an unauthenticated side channel. Ordering also matters: Q must be fixed before its response is computed, and a root containing R cannot be formed until the peer’s Q is available. The library supplies no scheduling or signature protocol that enforces these requirements.

Deterministic reconstruction. Once a position, its game history and the relevant seed have been authenticated, a verifier can recompute an expected query or set entry and compare it with the committed transcript. A full replay can check the path and facing against the map. Where the game provides a sound witness identifying a specific inconsistency, one point recomputation and a Merkle opening can avoid rebuilding an entire flight inside the referee. A membership opening alone cannot establish that an expected entry is absent, or that the whole set is correct; those cases need additional evidence and game-specific rules.

Determinism is necessary for this reconstruction but does not by itself bind an input, prove a seed was chosen honestly, or ensure that a complaint can be assembled. In particular, a mismatch under a newly supplied, unauthenticated seed justifies rejecting that reveal, not automatically convicting its sender for an earlier transcript. A subsequent dispute can attribute that mismatch only after the seed itself has been authenticated by the game’s accepted history.

Epoch disclosures. A disclosed seed allows anyone with the transcript to recompute the epoch’s scalars, undo its blinding, and enumerate the small public element universe. Privacy is therefore temporary. A game that needs to hide future positions must end or re-key the affected secret’s use before play resumes. Bounded re-keying needs an explicit terminal or reduced-privacy rule when the bound is reached; neither this library nor determinism supplies that rule. Deriving independent path and protocol secrets prevents a partial reveal of one secret from automatically disclosing the other.

Including the round in every dummy derivation prevents reuse of the same raw pad after a single round’s scalar has been exposed. With a fixed pad, recovered raw dummy points can be used in later chosen queries to test whether those points are still present. Their occupancy depends on set size. Round-dependent padding removes that particular persistent handle; it does not restore privacy after the seed itself is revealed. A single disclosed α_r likewise unblinds that round completely: $\alpha_r^{-1} \bmod \ell$ applied to S_r and Q_r , followed by a table lookup over the public universe, yields the round’s set, its position and the exact number of real entries, since the dummies are the entries that hit nothing.

Sampling and penalties. If an audit samples a round uniformly and independently after a fixed pattern of k deviations in n rounds, it detects a queried deviation with probability k/n . That statement is conditional. A deterministic hash over player-controlled commitments is not automatically an unbiased or unpredictable sample: grinding, reveal ordering and selective aborts need to be analysed. We make no unconditional catch-rate claim for the game.

Public evidence permits a penalty only if the rules admit that evidence, identify the responsible party soundly, and give an honest player time and standing to submit it. Players eliminated during play may still need to file before settlement. A later penalty cannot retract information already learned, and no stake size alone proves that cheating is irrational. This is accountability under explicit conditions, not general malicious-secure PSI.

4 Privacy and implementation limits

Leakage must be specified. The intended result is one membership bit per direction. Fixed-size padding hides the number of real entries from message length, but the transcript also exposes equality patterns. Duplicate inputs produce duplicate blinded points; Q_i can be compared directly with S_i to determine whether a party's own query occurs in its own set. For the sighting application that latter fact is normally public, since a tile sees itself. A general caller must account for it. The responses also expose equality across parties: $R_A = \alpha_A \alpha_B P_b$ and $R_B = \alpha_B \alpha_A P_a$ encode the same class exactly when $P_a = \pm P_b$, so any holder of the transcript learns whether $a = b$ without any secret; with several peers the same holds for every pair, since the two responses a pair exchanges are equal exactly when that pair holds the same element. The API accepts duplicates and does not enforce a game's input invariants.

Even correct membership bits reveal constraints on positions. Repeated negatives, map geometry, public actions and colluding observers can narrow the candidate set. Independent queries by several peers also change the oracle access available under one player's scalar. A two-party argument cannot be silently promoted to security against coalitions in a multi-player game.

A privacy proof is still required. Commutative DH matching has extensive security literature, but those proofs have exact protocol, group, encoding and adversary assumptions. Here a peer can submit an arbitrary encoded point and receive its multiplication by the responder's scalar. This is more than a passive transcript view. Describing DDH or one-more-DH as relevant assumptions is not a reduction for this implementation. We do not claim a simulation-based privacy theorem, including for the single-map x-only encoding, auxiliary inputs, or concurrent sessions.

Known discrete logarithms break the design. If $P_t = h_t G$ for a public nonzero scalar h_t , a requester knows the discrete log of its own query: $Q_A = (\alpha_A h_a) G$. From the response it obtains

$$K = (\alpha_A h_a)^{-1} R_B = \alpha_B G.$$

It can then enumerate $h_t K$ over the public universe and compare these values with S_B , recovering all real elements of B 's set. This works during an otherwise honest exchange and needs no guess about which set entry is real. The independent tests include this counterexample. It explains why replacing hash-to-point with a publicly known scalar times a base point is invalid; it does not prove the security of every alternative map.

Active inputs are not validated. A participant can submit any group element as its query (a chosen tile, the peer’s own published Q , or, with full coordinates, a sum of tile points) and compare the single response against both S and Q . Each response is therefore one chosen predicate over the responder’s set and position, not only one tile’s membership: $Q = P_x + P_y$ answers $[x \in V] \wedge [y \in V]$, and comparing the response with the responder’s own Q answers whether the two positions are equal. A participant can also answer with any point other than αQ , which forces the peer’s result to 0 while its own result stays correct, or fabricate a set. The raw reference ladder accepts arbitrary 32-byte inputs without a canonical-encoding or subgroup gate. An all-zero forged set and response make `petFinish` return 1, regardless of the claimed geometry; so do the non-canonical encodings of zero (p and 2^{255}), and given a zero response a single such entry among honest ones suffices. Low-order points on the curve or its twist force the result with probability 1/2 or 1/4 according to their order, so they are not harmless merely because their scalar leakage is limited; a validation gate must canonicalize before rejecting them. The reference library has no such gate; the current game refuses non-canonical and small-order second-lock and victim points at decode (§5). The correctness theorem excludes all of these inputs; deployments need their own validation and security argument. Hash commitments can authenticate malicious bytes just as readily as honest ones.

Execution and liveness. The implementation is not hardened against timing or memory-access observations: sorting and comparisons are data-dependent. Fixed loop counts for the ladder and finishing scan do not establish constant-time execution. The Merkle helpers share a scratch buffer, so callers must serialize their use; a concurrent call returns a wrong root without any error. The library has no timeout, reliable-delivery or fair-output mechanism, and no limit on how many queries a seed answers: `petRespond` is a pure function, so the caller must answer exactly one Q per seed, round and peer. These are obligations of the surrounding system, not consequences of the algebra.

5 Current game and verification

Dungeon Channel revision 5b050fc2 uses wire version 3 [8]. Its sight exchange no longer publishes the old $\alpha_i Q_j$ response. Instead, the watched player publishes sealed-envelope material from which an eligible watcher can obtain a read without notifying the watched player. The code retains a separate hearing response and uses additional secrets, flight fields and settlement evidence. This is a different protocol surface; its full correctness and privacy do not follow from the reference theorem.

At that revision the per-element primitive is unchanged: the scalar derivation, the element and pad encodings, the sort order and the Merkle leaf and node preimages are those of the reference, and `dungeon_curve.cpp` reproduces the fixture’s set block and query from the same 55 codes. The flight is not: it carries a 128-point hearing block and a 128-point second lock, its tree has depth 9 with the query at leaf 256 and the response leaf vacant, and the published second-lock and victim points are refused unless canonical and not of small order (`pointAcceptable`), a check the reference does not make; the set and hearing points are deliberately not gated. The fixture’s set is omnidirectional; the facing beam of $V(w, f)$ entered the game on 2026-08-18.

The current game retains the reconstruction pattern: its frontier binds a round’s query and flight root; closed-epoch records bind step histories and spent PET seeds; query and envelope disputes reopen that authenticated data. The terminal settlement code collects further reveal material before its filing phase. These mechanisms live in `dungeon_curve.cpp`, `dungeon_codec.hpp`

and `dungeon_core.cpp`. Their presence is an implementation fact, not a proof that every adversarial execution is covered.

Exact reference costs. The following counts follow from the standalone code. They exclude channel signatures, game state, transaction encoding and any hearing or envelope extension.

Quantity	Count
Flight 1, one party	4,128 bytes
Flight 2, one party	32 bytes
Both directions, both flights	8,320 bytes
Hash-to-point calls per party	129
Scalar multiplications outside hash-to-point per party	258
Merkle sibling path	256 bytes
Point, 16-bit index and sibling path	290 bytes

The 258 scalar multiplications are 129 for building, one for responding and 128 for finishing. Each hash-to-point also performs a cofactor-clearing ladder, giving 387 raw ladder calls per party in this uncached implementation. The 290-byte opening is not the size of a complete game dispute or transaction.

Platform limits. At Arcade platform revision `e20aff8f`, `engine/judge/wasm_judge.hpp` sets a per-call fuel cap of 500,000,000 and a state-output buffer of 16,384 bytes [9]. These are different constraints. A historical measurement exceeding the old 66.3M fuel limit does not establish that ZK verification is infeasible under the current cap. It also says nothing about every possible proof system or implementation. We report no current wall-clock, fuel or anonymity-set benchmark without a pinned executable, workload and reproducible measurement. The design trades in-round proving for later disclosure and adjudication; that trade-off remains meaningful without a universal cost-impossibility claim.

What the tests establish. `make test` checks RFC 7748 known answers with test-side clamping, sampled subgroup membership, the exhaustive distinctness of the element points, honest directional membership, boundary cases, the fixture of Appendix A, the duplicate and forged-zero examples, and indexed Merkle openings with tampered witnesses. `make verify` runs the independent BigInt implementation over the same fixture and examples and adds the known-discrete-log counterexample. The fixtures check selected derivations and serialized bytes. They cannot prove privacy, completeness of a paper, equivalence to the current game, or absence of implementation defects. The subgroup checks test nonzero points as well as $\ell P = O$ and $x((\ell - 1)P) = x(P)$. A zero output alone is ambiguous in an x-only ladder and would also accept an implementation returning zero for every input.

6 Related work and novelty boundary

Commutative private matching predates this work by decades, including Meadows (1986) and Huberman, Franklin and Hogg (1999) [1, 2]. Running membership in both directions is a specialization of that established construction. We claim no invention of the algebra, padding or hash-to-curve.

Commitment-and-verify defences for hidden game state predate the cryptographic-PSI line. Baughman and Levine commit to each step’s decision with a one-way hash and give a commutative-encryption test for whether two players share a cell without revealing their positions [10]; Chambers et al. detect map hacks in peer-to-peer real-time strategy play by bit commitment and post-game verification [11].

OpenConflict applied private set intersection with chaff padding to fog-of-war state distribution in real-time games in 2011 [12]. Its proofs assume passive players; its discussion section already outlines signing all traffic, periodic signed commitments to actions, end-of-game disclosure to a verifying server, and committing to and later revealing the seed of the generator used in the set-intersection protocol, left there as future work. Narayanan et al.’s private proximity work treats asymmetric outputs, repeated invocation and online guessing by a malicious friend, not only honest-but-curious parties [13]. Neither private encounters nor directionality is a novelty claim here.

Thomson’s 2020 article proposed private set intersection for sightings, salted position commitments on a blockchain and end-of-game disclosure of positions and actions for deterministic replay, and pointed to Xaya’s game channels as the setting [14]. His publicly available PSI implementation was the practical starting point of this work [15]; that repository has since added a dispute layer which credits this design. The high-level combination of PSI and retrospective game checking therefore predates this artifact, and the seed-committed blinding that makes a transcript recomputable is OpenConflict’s outline. Our contribution is the particular deterministic transcript format and its implementation with game adjudication, not the general idea of replaying a hidden game.

Zero-knowledge proofs per action are the other road. Dark Forest hides positions behind hash commitments and attaches a zkSNARK validity proof to each on-chain action [19]; Ingonyama’s verifiable committed oblivious transfer is a later proof-based construction for the same mechanic [20]. This design replaces in-round proving with disclosure and adjudication; §5 states what that trade does and does not establish about cost.

Detection and penalties have their own literature. Aumann and Lindell formalize covert security; Asharov and Orlandi add publicly verifiable evidence of cheating [16, 17]. Faust et al.’s financially backed covert security already supports blockchain adjudication with checks reduced to a protocol message or, interactively, an instruction [18]. Thus bounded checking and financial accountability are not themselves new. Our system also has not been proved to realize those formal security definitions.

Game channels provide the signed off-chain execution and dispute setting [21]. This artifact combines existing ideas under concrete execution and disclosure constraints. The comparison supports presenting it as an engineering case study with a conditional correctness proof. Establishing a stronger research novelty claim would require a precisely stated new theorem or demonstrated capability beyond these predecessors, with a corresponding proof or reproducible evaluation. That claim is not established here.

Acknowledgments

Edward Thomson’s publicly available implementation was the practical starting point of this work. The cryptographic constructions and game-channel framework retain the attribution above. Code is credited to Xaya Developers; the paper’s named authors are listed separately.

A The fixture

Seed bytes A0, A1, ..., BF (byte i is A0 + i); round 7; own element 10897, the Morton code of tile (120, 5); the 55 supplied codes, in the enumeration order the game produced them (the order does not affect the flight): 10790, 10796, 10798, 10884, 10886, 10892, 10894, 10791, 10797, 10799, 10885, 10887, 10893, 10895, 10802, 10808, 10810, 10896, 10898, 10904, 10906, 10777, 10779, 10801, 10803, 10809, 10811, 10897, 10899, 10905, 10907, 10806, 10812, 10814, 10900, 10902, 10908, 10910, 10807, 10813, 10815, 10901, 10903, 10909, 10911, 10850, 10856, 10858, 10944, 10946, 10952, 10954, 10945, 10948, 10949. Expected values, hexadecimal:

SHA-256 of flight 1 (4,128 bytes)	106e8a31ba80abcab999bd08fd7637eb a04c87232637416d74d325f0b541b7d9
Q	2ec31f4c132caefd3a05ae1a5fc40974 437417b36dd7efde2e6d493c4d551417
$R = \alpha Q$ (the self-response)	fc91185d3b6994fda37920ebe2473849 321249ca4d345bfedffde2319a73b12b
Merkle root over flight 1 and R	81fdb4d86cb4be4dfc44adc3822dc9f ce9bb06026316c8eedd3a6e9e1448fca

The game's own test suite at the fixture revision pinned the same root and the same first sibling of R over these inputs. Since the own element is in the supplied set, `petFinish` returns 1.

References

- [1] C. Meadows. A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party. *IEEE Symposium on Security and Privacy*, 1986.
- [2] B. A. Huberman, M. Franklin and T. Hogg. Enhancing privacy and trust in electronic communities. *ACM EC*, 1999.
- [3] D. J. Bernstein. Curve25519: new Diffie–Hellman speed records. *PKC*, 2006.
<https://cr.yp.to/ecdh/curve25519-20060209.pdf>.
- [4] D. J. Bernstein, M. Hamburg, A. Krasnova and T. Lange. Elligator: elliptic-curve points indistinguishable from uniform random strings. *ACM CCS*, 2013.
<https://elligator.cr.yp.to/elligator-20130828.pdf>.
- [5] A. Langley, M. Hamburg and S. Turner. Elliptic curves for security. RFC 7748, 2016.
<https://www.rfc-editor.org/rfc/rfc7748>.
- [6] A. Faz-Hernandez et al. Hashing to elliptic curves. RFC 9380, 2023.
<https://www.rfc-editor.org/rfc/rfc9380>.
- [7] Xaya Developers. Fog-of-war reference implementation and known-answer tests.
<https://github.com/xaya/fog-of-war>.
- [8] Xaya Developers. Dungeon Channel, revision 5b050fc24d76b45a97faa3ab8d8444d4d66c1718.
<https://github.com/xaya/arcade-dungeonchannel> (private repository; access on request). The fixture revision 1ad3b7fcec7f9b51221720e9de238cbdfbd204c8 (2026-08-01) lies outside its retained history.

- [9] Xaya Developers. Arcade platform, revision e20aff8f723a8d9fbdcfcf32d12fc5bbb3dc600e. <https://github.com/xaya/arcade-platform> (private repository; access on request).
- [10] N. E. Baughman and B. N. Levine. Cheat-proof payout for centralized and distributed online games. *IEEE INFOCOM*, 2001.
- [11] C. Chambers, W. Feng, W. Feng and D. Saha. Mitigating information exposure to cheaters in real-time strategy games. *ACM NOSSDAV*, 2005. https://www.thefengs.com/wuchang/cstrike/nossdav05_mitigating.pdf.
- [12] E. Bursztein, M. Hamburg, J. Lagarenne and D. Boneh. OpenConflict: preventing real time map hacks in online games. *IEEE Symposium on Security and Privacy*, 2011. <https://crypto.stanford.edu/~dabo/pubs/papers/onlinegames.pdf>.
- [13] A. Narayanan, N. Thiagarajan, M. Lakhani, M. Hamburg and D. Boneh. Location privacy via private proximity testing. *NDSS*, 2011. <https://crypto.stanford.edu/~dabo/papers/locpriv.pdf>.
- [14] E. A. Thomson. Preventing cheaters in fog of war games. June 2020. <https://edward-thomson.medium.com/preventing-cheaters-in-fog-of-war-games-69f202fbe107>.
- [15] E. A. Thomson. Private-Set-Intersection, commit 350ac6a (2025-10-26), accessed July 2026. <https://github.com/EdwardAThompson/Private-Set-Intersection>. Later commits adopt material from this paper.
- [16] Y. Aumann and Y. Lindell. Security against covert adversaries: efficient protocols for realistic adversaries. *TCC*, 2007. <https://iacr.org/archive/tcc2007/43920137/43920137.pdf>.
- [17] G. Asharov and C. Orlandi. Calling out cheaters: covert security with public verifiability. *ASIACRYPT*, 2012. https://doi.org/10.1007/978-3-642-34961-4_41.
- [18] S. Faust, C. Hazay, D. Kretzler and B. Schlosser. Financially backed covert security. *PKC*, 2022. <https://eprint.iacr.org/2021/1652>.
- [19] Dark Forest team. Announcing Dark Forest, 2020. <https://blog.zkg.me/announcing-darkforest>.
- [20] Ingonyama. Cryptographic fog of war, March 2024. <https://www.ingonyama.com/post/cryptographic-fog-of-war>.
- [21] D. Kraft. Game channels for trustless off-chain interactions in decentralized virtual worlds. *Ledger* 1:84–98, 2016. <https://doi.org/10.5195/ledger.2016.15>.